
UNIDAD 05

Bucles

Programación en Computación · Ingeniería Electromecánica — 2.º año · UTN FR Reconquista

Longhi Pablo · Talijancic Iván

Qué vamos a ver

01 `for` — cuando sabés cuántas veces

02 `while` y `do-while`

03 La **traza**: cómo se depura un bucle

04 Los tres **patrones**: acumulador, contador, máximo

05 Bucles **infinitos** y cómo salir

Lo de hoy es la base de la unidad 07. Cuando aparezcan los vectores, se recorren con **estos mismos bucles y estos mismos patrones**.

PARTE 1

| El problema

Tres mediciones se bancan

src/app.js

```
console.log('Medición 1')  
console.log('Medición 2')  
console.log('Medición 3')
```

Funciona perfecto. Tres líneas, tres mediciones.

Las de todo el mes, no

src/app.js

```
console.log('Medición 1')  
console.log('Medición 2')  
// ... 28 líneas más ...  
console.log('Medición 31')
```

Inviabile. Y el problema de fondo todavía no apareció.

PREGUNTA

**¿Y si el usuario decide
cuántas mediciones son?**

No podés escribir a mano una cantidad de líneas que se conoce recién cuando el programa corre.

Tres líneas, treinta y una vueltas

src/app.js

```
for (let i = 1; i <= 31; i++) {  
  console.log(`Medición ${i}`)  
}
```

Cambiar el 31 por 365 no agrega una sola línea.

Las tres partes

src/app.js

```
for (let i = 1; i <= 3; i++) {  
  //      ↑      ↑      ↑  
  //      |      |      └ incremento: al final de cada vuelta  
  //      |      └ condición: mientras sea true, sigue  
  //      └ inicialización: UNA vez, al principio  
}
```

Ese **i** es la **variable de control**, por *índice*. Es la única variable del curso a la que se le permite un nombre de una letra.

El orden exacto

01 **Inicialización** — una sola vez

02 **¿Condición?** Si es `false`, el bucle termina

03 **Cuerpo** — las líneas entre llaves

04 **Incremento**

05 Volver al paso 2

La condición se evalúa **antes** de entrar. Si da `false` la primera vez, el cuerpo **no se ejecuta nunca**.

Seguirlo paso a paso

VUELTA	I	I <= 4	IMPRIME	I++
1	1	sí	1	2
2	2	sí	2	3
3	3	sí	3	4
4	4	sí	4	5
—	5	NO	—	corta

```
for (let i = 1;  
      i <= 4;  
      i++) {  
  console.log(i)  
}
```

Con `i = 5` la condición da `false` y el cuerpo **no se ejecuta**.

Hacela en papel

Dos minutos de tabla te ahorran veinte de mirar la pantalla

Cuando un bucle no hace lo que esperás, **no lo mires: trazalo**. Una columna por variable, una fila por vuelta.

De cuánto en cuánto

```
for (let i = 0; i <= 20; i += 5) {  
  console.log(i)  
}
```

SALIDA

0 · 5 · 10 · 15 · 20

```
for (let i = 5; i >= 1; i--) {  
  console.log(i)  
}
```

SALIDA

5 · 4 · 3 · 2 · 1

El incremento no tiene que ser de a uno, ni tiene que ir para arriba.

PARTE 3

| while y do-while

Las mismas tres piezas, repartidas

src/app.js

```
let i = 1           // 1. inicialización – ANTES

while (i <= 3) {    // 2. condición
  console.log(i)
  i++              // 3. incremento – ADENTRO
}
```

ERROR FRECUENTE

Ese `i++` es **obligatorio**. Sin él, `i` vale 1 para siempre, la condición nunca se hace falsa y el programa **no termina nunca**.

Al menos una vez

✗ WHILE

```
let x = 50

while (x > 100) {
  console.log('no entra')
}
```

Condición falsa de entrada: **no imprime nada.**

✓ DO-WHILE

```
let x = 50

do {
  console.log('entra una vez')
} while (x > 100)
```

La condición se evalúa **al final.**

Para qué sirve: el centinela

src/app.js

```
let temperature

do {
  temperature = parseFloat(prompt('Temperatura (0 para terminar): '))
} while (temperature !== 0)
```

Primero tenés que **pedir** el dato; recién después podés saber si sirve.

Ese `0` es el **centinela**: un valor acordado que significa «no hay más datos». Fijate que la variable se declara **antes** del `do`.

Los tres, comparados

SITUACIÓN

BUCLE

Sé cuántas veces (10 mediciones, del 1 al 31)

`for`

Depende de algo que cambia (mientras la presión baje)

`while`

Hay que ejecutarlo al menos una vez (validar, centinela)

`do-while`

Los tres son **intercambiables**. La elección es de **legibilidad**, no de capacidad.

EN LA DUDA, `FOR`: EL INCREMENTO ESTÁ A LA VISTA.

PARTE 4

| Los tres patrones

El más importante del curso

src/app.js

```
let total = 0 // 1. ANTES: arranca en cero

for (let i = 1; i <= 5; i++) {
  total = total + i // 2. ADENTRO: se le suma algo
}

console.log(total) // 3. DESPUÉS: el resultado
```

SALIDA

15

Por qué funciona

VUELTA	I	TOTAL ANTES	TOTAL = TOTAL + I
1	1	0	1
2	2	1	3
3	3	3	6
4	4	6	10
5	5	10	15

Cada vuelta arranca del total de la anterior. **Por eso tiene que sobrevivir entre vueltas.**

PREGUNTA

¿Y si declaro el acumulador adentro del bucle?

```
for (...) { let total = 0; total = total + i }
```

El acumulador va afuera

× ASÍ NO

```
for (let i = 1; i <= 5; i++) {  
  let total = 0  
  total = total + i  
}
```

Nace y muere en cada vuelta. Arranca en 0 cinco veces.

✓ ASÍ SÍ

```
let total = 0  
  
for (let i = 1; i <= 5; i++) {  
  total = total + i  
}
```

Sobrevive entre vueltas, y existe al salir.

Bucle + condicional

src/app.js

```
let outOfRange = 0
let measured = 0

for (let i = 1; i <= 5; i++) {
  measured = 350 + i * 15

  if (measured < 361 || measured > 399) {
    outOfRange++
  }
}
```

SALIDA

```
365 · 380 · 395 · 410 · 425
Fuera de rango: 2
```


Guardar el campeón

src/app.js

```
let maximum = 0
let measured = 0

for (let i = 1; i <= 5; i++) {
  measured = 350 + i * 15
  if (measured > maximum) {
    maximum = measured
  }
}
```

SALIDA

Máxima: 425

Anda. ¿Siempre?

PREGUNTA

**Mediciones: -5, -12,
-19. ¿Cuál es el máximo?**

Con `let maximum = 0`, ¿qué contesta el programa?

El cero que no era del conjunto

SALIDA

```
Máxima (arrancando en 0):  0 °C  
Máxima (bien):           -5 °C
```

ERROR FRECUENTE

Ese `0` **no es ninguna de las mediciones**. Nunca hubo un `-5 > 0`, así que el campeón inicial nunca fue reemplazado.

Inicializá con el **primer valor real**, no con un número inventado.

Los tres, de memoria

los tres patrones

```
let total = 0 // ACUMULADOR
for (...) { total = total + valor }

let cuantos = 0 // CONTADOR
for (...) { if (cond) { cuantos++ } }

let maximo = primerValor // MÁXIMO
for (...) { if (v > maximo) { maximo = v } }
```

Los tres tienen la misma forma:
declarar antes · actualizar adentro · usar después

PARTE 5

| Cuando no termina

El bucle que no termina

src/app.js

```
let i = 1

while (i <= 5) {
  console.log(i)
  // falta el i++
}
```

Imprime 1 para siempre.

Para salir: Ctrl + C

Las tres causas

- 01 **Falta el incremento** — `while` sin `i++`
- 02 **Va para el lado equivocado** — `i--` con la condición `i <= 10`
- 03 **La condición no puede ser falsa** — `while (i > 0)` con `i` que sólo crece

Si el programa imprime miles de líneas o queda colgado sin devolver el cursor: **Ctrl + C** y **revisá el incremento**. No está roto Node, está roto el bucle.

Todo junto

src/app.js

```
const howMany = parseInt(prompt('¿Cuántas mediciones? '))
let total = 0
let measured = 0

for (let i = 1; i <= howMany; i++) {
  measured = parseFloat(prompt(`Medición ${i} [V]: `))
  total = total + measured
}

console.log(`Promedio: ${(total / howMany).toFixed(2)} V`)
```

El bucle **no sabe** de antemano cuántas vueltas va a dar. Eso es lo que no se podía hacer copiando y pegando.

Lo que hay que llevarse

01 `for` si sabés cuántas · `while` si depende · `do-while` si va al menos una.

02 Cuando algo no cierra: **hacé la traza**.

03 Acumulador, contador y máximo: **declarar antes, actualizar adentro, usar después**.

04 El máximo arranca con el **primer valor real**, no en cero.

05 ¿No termina? `Ctrl + C` y mirá el incremento.

PRÓXIMA CLASE • UNIDAD 06

Funciones

Parámetros • Valor de retorno • `Math.random()`

Los programas de hoy, en la mitad de líneas.